Machine Learning For Networking

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Project 8 – DDoS attacks detection and characterization **3**
1.1 Di cosa parla il progetto . . . 3
1.1.1 Il problema . . . 3
1.1.2 Il dataset . . . 3
1.1.3 Cosa chiede il progetto, in una frase per sezione . . . 4
1.2 Setup comune (da fare tutti insieme, prima di dividersi) . . . 4
1.3 Sezione 1 – Data exploration e pre-processing . . . 5
1.3.1 Cosa chiede . . . 5
1.3.2 Investigazione del dataset (punto 1.1 della traccia) . . . 5
1.3.3 Pre-processing (punto 1.2 della traccia) . . . 6
1.3.4 Feature matrix finale (punto 1.3 della traccia) . . . 7
1.4 Sezione 2 – Classificazione supervisionata . . . 7
1.4.1 Train/test split (punto 2.1 della traccia) . . . 7
1.4.2 Training e valutazione (punto 2.2 della traccia) . . . 7
1.4.3 Analisi dei risultati (punto 2.3 della traccia) . . . 8
1.5 Sezione 3 – Clustering . . . 9
1.5.1 Cosa chiede . . . 9
1.6 Sezione 4 – Spiegabilità dei cluster e anomaly detection . . . 10
1.6.1 Distribuzione delle classi nei cluster (punto 4.1 della traccia) . 10
1.6.2 Anomaly detection su solo traffico benigno (punto 4.2 della traccia) . . . 11
1.7 Divisione del lavoro in 4 persone . . . 12
1.7.1 Logica della divisione . . . 12
1.7.2 Ruoli . . . 12
1.7.3 Timeline indicativa . . . 12
1.7.4 Regole per lavorare bene insieme . . . 13
1.8 Checklist finale . . . 13

Chapter 1

Project 8 – DDoS attacks detection and characterization

1.1 Di cosa parla il progetto

1.1.1 Il problema

Un attacco **DDoS** (*Distributed Denial of Service*) cerca di rendere un servizio inutilizzabile per gli utenti legittimi saturandone le risorse (banda, CPU, tabelle di connessione) con traffico generato da molte macchine compromesse. Gli attacchi possono avvenire a livello rete, trasporto o applicazione, usando protocolli come TCP, UDP, ICMP e HTTP.

Il dataset contiene soprattutto attacchi **riflessivi/amplificati** (*reflection/amplification*): l'attaccante invia piccole richieste a server pubblici (DNS, NTP, LDAP, SNMP, NetBIOS, MSSQL, ...) falsificando l'IP sorgente (*IP spoofing*) con quello della vittima. I server rispondono alla vittima con risposte molto più grandi della richiesta: il fattore di amplificazione moltiplica il volume di traffico. A questi si aggiungono attacchi "di esaurimento" come **SYN flood** (riempie la coda delle connessioni TCP semiaperte) e **UDP flood / UDP-Lag**.

L'ipotesi di fondo del progetto è che *attacchi diversi lascino impronte statistiche diverse* nelle feature di flusso (dimensione dei pacchetti, rate, inter-arrival time, flag TCP, porte, ...), e che quindi il Machine Learning possa (i) riconoscerli e (ii) raggrupparli.

1.1.2 Il dataset

- Ogni riga è un **flusso**: sequenza di pacchetti tra una coppia (IP sorgente, IP destinazione) con porte e protocollo, estratto con **CICFlowMeter-V3**. La struttura (feature, tipologie di attacco, 25 utenti simulati su HTTP/HTTPS/FTP/SSH/email) corrisponde a quella del dataset pubblico **CIC-DDoS2019** dell'University of New Brunswick: conviene leggerne il paper per capire come è stato costruito.
- Circa 80+ feature numeriche: statistiche di lunghezza pacchetti, inter-arrival time (IAT), rate, flag TCP, feature di bulk, subflow, active/idle time.
- **Ground truth**: colonna `label` con 12 classi: `benign` + 11 attacchi

(`ddos_ldap`, `ddos_udp`, ...). L'elenco esatto va verificato con `value_counts()`.
- Dimensione: ∼28 MB di CSV, quindi gestibile interamente in RAM con pandas; solo alcune operazioni (silhouette, t-SNE, OC-SVM) richiedono subsampling.

1.1.3 Cosa chiede il progetto, in una frase per sezione

1. **Esplorazione e pre-processing**: capire i dati e costruire una feature matrix pulita.
2. **Classificazione (supervisionata)**: dato un flusso, predire il tipo di attacco; confrontare split casuale e split temporale.
3. **Clustering (non supervisionato)**: senza usare le label, vedere se i flussi si raggruppano in "famiglie".
4. **Spiegabilità e anomaly detection**: usare le label *a posteriori* per interpretare i cluster; infine addestrare un modello solo su traffico benigno e vedere quali attacchi emergono come anomalie.

Attenzione

Il filo conduttore di tutto il progetto è il **data leakage**. Feature come `Flow ID`, `Source IP`, `Destination IP`, `Timestamp` (e in parte `Source Port`) identificano *come* è stato registrato il dataset, non *come* si comporta l'attacco. Nel CIC-DDoS2019 l'attaccante e la vittima hanno IP fissi: un modello che usa gli IP raggiunge accuratezze altissime senza aver imparato nulla di utile. Vanno rimosse dalla feature matrix (ma conservate a parte: servono per l'EDA, per lo split temporale e per l'analisi degli errori).

1.2 Setup comune (da fare tutti insieme, prima di dividersi)

Strategia operativa

- **Repository Git** con struttura: `data/` (fuori dal versionamento), `notebooks/`, `src/` (funzioni condivise), `figures/`, `report/`.
- Un modulo `src/preprocessing.py` con *una sola* funzione `load_clean()` che tutti importano: così nessuno lavora su una versione diversa dei dati.
- Salvare il dataset pulito in `.parquet` (lettura molto più veloce del CSV, mantiene i tipi).
- Fissare `RANDOM_STATE = 42` ovunque (split, modelli, subsampling) per riproducibilità.
- Concordare uno **stile unico per i grafici** (palette per classe fissa: `benign` sempre dello stesso colore in tutti i plot) e un file `src/plotting.py`.
- Stabilire il formato di consegna richiesto dal docente (notebook, report, presentazione) e chiarire i dubbi con il referente indicato nella traccia.

1.3 Sezione 1 – Data exploration e pre-processing

1.3.1 Cosa chiede

Presentare il dataset con visualizzazioni e statistiche, a livello di traffico generico e a livello di classe (GT); poi pulire, analizzare correlazioni, fare PCA, decidere scaling ed encoding, e caratterizzare la feature matrix finale.

1.3.2 Investigazione del dataset (punto 1.1 della traccia)

Strategia operativa

Controlli di qualità iniziali

- `df.shape`, `df.dtypes`, `df.isna().sum()`, duplicati (`df.duplicated().sum()`).
- Valori `inf`: tipici in `Flow Bytes/s` e `Flow Packets/s` quando `Flow Duration`=0.
- Colonne costanti (varianza nulla): nel CIC-DDoS2019 molte feature di bulk e flag (`Bwd PSH Flags`, `Fwd/Bwd URG Flags`, `*Avg Bytes/Bulk`, ...) sono sempre 0.
- Colonne duplicate: `Fwd Header Length.1` è una copia di `Fwd Header Length`; verificare anche altre coppie identiche (es. `Subflow Fwd Packets` vs `Total Fwd Packets`).
- Eventuale colonna indice `Unnamed: 0` da eliminare.
- Valori negativi impossibili (es. `Init_Win_bytes` $= -1$ significa "non disponibile").

Livello traffico generico

- Numero di flussi totale e per classe (bar plot, scala log se sbilanciato): evidenziare lo **sbilanciamento delle classi**.
- Numero di IP sorgente/destinazione distinti, flussi per IP (ECDF), top-k IP.
- Distribuzione dei protocolli (TCP=6, UDP=17, ...) globale e per classe.
- Andamento temporale: flussi per minuto colorati per label. Si vede che gli attacchi sono eseguiti in finestre temporali successive (fondamentale per capire la Sezione 2.1b).

Livello GT (per classe)

- ECDF delle **porte destinazione e sorgente** per classe: gli attacchi riflessivi usano porte di servizio note (DNS 53, NTP 123, SNMP 161, NetBIOS 137–139, LDAP 389, MSSQL 1434, SSDP 1900, ...).
- ECDF/boxplot (asse log) di `Flow Duration`, `Total Fwd Packets`, `Fwd Packet Length Mean`, `Flow Bytes/s`, `Flow IAT Mean` per classe.
- Tabella con mediana e IQR di alcune feature per classe.
- Flag TCP (SYN, ACK, ...) per classe: SYN flood deve emergere chiaramente.

Domande a cui il report deve rispondere

Si vede la differenza tra traffico benigno e attacchi? Quali attacchi sono facili da distinguere tra loro e quali si sovrappongono (es. attacchi riflessivi UDP con pacchetti di dimensione simile)? Questa risposta anticipa le confusioni che vedrete nella Sezione 2.

1.3.3 Pre-processing (punto 1.2 della traccia)

Strategia operativa

Pulizia e selezione

1. Rimuovere identificatori: `Flow ID`, IP, `Timestamp` (tenuto a parte), `Source Port` (porte effimere casuali = rumore e possibile leakage).
2. Sostituire `inf` con `NaN`, poi imputare o eliminare le righe (giustificare: se sono poche, eliminarle).
3. Eliminare colonne costanti e duplicate.
4. **Correlazione**: heatmap di Pearson (e Spearman, più adatta a distribuzioni heavy-tailed). Eliminare una feature per ogni coppia con $|\rho| > 0.95$ (o soglia giustificata). Molte feature sono derivate l'una dall'altra (mean/max/std, totali/subflow).

PCA

- Va fatta *dopo* lo scaling (la PCA è sensibile alla scala).
- Plot della varianza spiegata cumulata: quante componenti per il 90/95%?
- Scatter PC1 vs PC2 colorato per label: prima intuizione della separabilità.
- Loadings delle prime componenti: quali feature pesano di più?
- Decisione: per la classificazione con alberi conviene *non* usare la PCA (si perde interpretabilità, servono le feature originali per l'analisi FP/FN); per clustering e anomaly detection può aiutare (distanze più significative in dimensione ridotta). Motivarlo nel report.

Scaling ed encoding

- Le feature sono **heavy-tailed** (byte/s da 0 a 10^9): applicare `log1p` alle feature non negative prima dello scaling è spesso più efficace di qualunque scaler.
- `StandardScaler` (o `RobustScaler`, meno sensibile agli outlier) necessario per KNN, SVM, regressione logistica, k-means, PCA; *non* necessario per alberi/random forest (invarianti a trasformazioni monotone). Spiegarlo.
- `Protocol` è categorica (6/17/0): **one-hot**, perché un valore numerico implicherebbe un ordine inesistente.
- `Destination Port`: categorica ad alta cardinalità. Opzioni: raggruppare in bucket (well-known < 1024, registered, dynamic) e fare one-hot, oppure flag binari per le porte più frequenti. Discutere il rischio che la porta sia una scorciatoia che rende il problema banale.
- `label`: `LabelEncoder` per i classificatori (one-hot non serve per i modelli sklearn).

Attenzione

Scaler, PCA e qualunque trasformazione "appresa" vanno **fittati solo sul training set** e poi applicati al test. Usare una `sklearn.pipeline.Pipeline` risolve il problema automaticamente, anche dentro la cross-validation.

1.3.4 Feature matrix finale (punto 1.3 della traccia)

Ripetere per la matrice finale: ECDF/EPDF delle feature selezionate (dopo log e scaling), heatmap di correlazione residua, dimensione finale (n righe $\times$ d feature) e lista delle feature tenute con motivazione.

1.4 Sezione 2 – Classificazione supervisionata

1.4.1 Train/test split (punto 2.1 della traccia)

Strategia operativa

a) Split per-flow stratificato: `train_test_split` con `test_size=0.2`, `stratify=y` e `random_state=42`. Ogni flusso è trattato come indipendente.
b) Split temporale stratificato: per ogni classe, ordinare per `Timestamp` e prendere il primo 80% come train e il restante 20% come test.

```
def temporal_split(df, label_col="label", time_col="Timestamp", test_size=0.2):
    df = df.copy()
    df[time_col] = pd.to_datetime(df[time_col])
    train_idx, test_idx = [], []
    for _, g in df.groupby(label_col):
        g = g.sort_values(time_col)
        cut = int(len(g) * (1 - test_size))
        train_idx.extend(g.index[:cut])
        test_idx.extend(g.index[cut:])
    return df.loc[train_idx], df.loc[test_idx]
```

Verificare con un plot che, per ogni classe, il massimo timestamp del train sia minore del minimo del test.

1.4.2 Training e valutazione (punto 2.2 della traccia)

Strategia operativa

Scelta dei modelli (almeno 2, gli stessi per entrambi gli split). Suggerimento: uno "semplice" e uno "forte", così il confronto è interessante.

- **Random Forest**: robusto, non richiede scaling, fornisce feature importance (utile per spiegare gli errori).
- **KNN** o **Logistic Regression**: baseline sensibile allo scaling, contrasto utile.

- Opzionali: Decision Tree (interpretabile), XGBoost/LightGBM, MLP.

Valutazione

- Accuracy su train *e* test (serve per diagnosticare overfitting).
- `classification_report` e confusion matrix normalizzata per riga (recall per classe).
- Con classi sbilanciate l'accuracy è fuorviante: usare **macro-F1** come metrica principale.
- Misurare il **tempo di training e di inferenza** (`time.perf_counter()`): serve per il punto c).

b) Over/underfitting: train $\approx$ 100% e test molto più basso = overfitting (tipico di alberi senza limite di profondità). Rimedi: `max_depth`, `min_samples_leaf`, meno feature, più regolarizzazione. Train e test entrambi bassi = underfitting (modello troppo semplice o feature non informative).

c) Hyper-parameter tuning: `GridSearchCV` o `RandomizedSearchCV` con `StratifiedKFold(5)` e `scoring="f1_macro"`. Esempi di griglie:

- RF: `n_estimators` $\in \{50, 100, 300\}$, `max_depth` $\in \{\text{None}, 10, 20\}$, `min_samples_leaf` $\in \{1, 5\}$, `class_weight` $\in \{\text{None}, \text{balanced}\}$.
- KNN: `n_neighbors` $\in \{3, 5, 11, 21\}$, `weights`, `metric`.

Per lo split temporale la CV dovrebbe rispettare il tempo (es. fold ordinati temporalmente per classe, o `TimeSeriesSplit`): discuterlo. Riportare una tabella *performance vs tempo di training* e scegliere il compromesso.

Domande a cui il report deve rispondere

Quale modello è migliore e perché? Perché alcune classi hanno performance più basse? (Risposta attesa: attacchi riflessivi su UDP con pattern di traffico quasi identici, es. DNS/LDAP, NetBIOS/Portmap, UDP/UDP-Lag, generano flussi molto simili e vengono confusi tra loro; attacchi con firma netta come SYN sono facili.)

1.4.3 Analisi dei risultati (punto 2.3 della traccia)

Strategia operativa

a) FP e FN

- Estrarre i campioni mal classificati e identificare le coppie di classi più confuse dalla confusion matrix.
- Per una coppia (vera A, predetta B): confrontare le distribuzioni di feature (ECDF/boxplot) di tre gruppi: A corretti, B corretti, A predetti come B. Tipicamente i mal classificati "sembrano" B nelle feature più importanti.
- Feature importance (RF) o permutation importance per scegliere quali feature mostrare.
- Guardare anche gli identificatori rimossi (IP, timestamp) dei campioni sbagliati: sono flussi di bordo all'inizio/fine di un attacco? Flussi benigni finiti in mezzo a una finestra di attacco?

b) Metriche nel contesto reale

- Riformulare il problema come binario (benign vs attacco).
- **FP** = traffico benigno segnalato come attacco → *falso allarme*: l'operatore perde tempo o, peggio, blocca utenti legittimi. Lo misura la **precision** della classe attacco (equivalentemente il FPR / la recall della classe benign).
- **FN** = attacco classificato come benigno → *attacco non rilevato*. Lo misura la **recall** (detection rate) della classe attacco.
- Trade-off: in un SOC un FPR alto causa *alert fatigue*; una recall bassa lascia passare attacchi lenti/subdoli.
- Nel multiclasse, confondere due tipi di attacco è meno grave che confondere attacco e benigno (la mitigazione resta attivata): evidenziarlo.

c) Per-flow vs per-time: lo split per-flow tende a essere **ottimistico**, perché flussi quasi identici e consecutivi dello stesso attacco finiscono sia in train sia in test (quasi-duplicati → leakage). Lo split temporale simula lo scenario reale (si addestra sul passato, si predice il futuro) e cattura il *concept drift* all'interno di un attacco (cambi di intensità/fase). Se lo split temporale va peggio, è la misura più onesta.

1.5 Sezione 3 – Clustering

1.5.1 Cosa chiede

Raggruppare i flussi **senza label**, con almeno 2 algoritmi e almeno 2 metriche di similarità, determinare il numero di cluster, fare tuning, valutare e visualizzare.

Strategia operativa

Input: feature matrix della Sezione 1 (log + scaling, eventualmente PCA). *La label non deve entrare in nessun passaggio di fit.*

Subsampling: silhouette ($O(n^2)$), agglomerative e t-SNE sono costosi; lavorare su un campione stratificato (es. 10–20k flussi) per la scelta degli iperparametri e dichiararlo.

Algoritmi suggeriti (sceglierne almeno 2 con logiche diverse):

- **K-Means**: centroidi, cluster convessi, serve k.
- **DBSCAN / HDBSCAN**: densità, trova forme arbitrarie e rumore, non serve k.
- **Agglomerative clustering**: gerarchico, dendrogramma utile per vedere "famiglie" di attacchi, supporta più metriche.
- **Gaussian Mixture**: soft clustering, cluster ellittici.

1. Numero di cluster

- K-Means: elbow (inertia vs k) + silhouette vs k, per $k = 2 \dots 20$. Non forzare $k = 12$: il punto è vedere se emergono meno "famiglie" delle classi.
- DBSCAN: *k-distance plot* per scegliere `eps`, `min_samples` $\approx 2 \cdot d$.
- Agglomerative: dendrogramma + silhouette.
- GMM: BIC/AIC vs numero di componenti.

2. **Metriche di similarità**
   - **Euclidea**: dipende dalla magnitudine (volume del flusso).
   - **Coseno**: dipende dalla "forma" del vettore (proporzioni tra feature), non dalla scala.
   - Note pratiche: K-Means in sklearn supporta solo l'euclidea; per il coseno normalizzare L2 i vettori e poi K-Means (*spherical k-means*). Agglomerative (`linkage="average"`) e DBSCAN accettano `metric="cosine"`. Opzionale: Manhattan.
3. **Tuning, valutazione, visualizzazione**
   - Metriche interne (non usano la GT): silhouette, Davies–Bouldin, Calinski–Harabasz.
   - Metriche esterne (usano la GT *solo per valutare*): ARI, NMI, homogeneity, completeness, purity.
   - Visualizzazione: PCA 2D e t-SNE/UMAP su un campione, due pannelli affiancati: colore = cluster e colore = label vera.
   - Silhouette plot per cluster (stile "coltelli").
4. **Analisi grezza**: dimensione dei cluster (bar plot), ECDF del numero di flussi per cluster, silhouette media per cluster, quanti punti DBSCAN etichetta come rumore.

1.6 Sezione 4 – Spiegabilità dei cluster e anomaly detection

1.6.1 Distribuzione delle classi nei cluster (punto 4.1 della traccia)

Strategia operativa

- **Tabella di contingenza** cluster × label (`pd.crosstab`), mostrata come heatmap normalizzata per riga (composizione del cluster) e per colonna (dove finisce ogni classe).
- **a)** Per ogni classe, contare in quanti cluster cade (considerando solo i cluster che contengono almeno, es., l'1% dei flussi di quella classe, per ignorare la coda) e farne l'**ECDF**.
- **b)** Per ogni cluster: purity (frazione della classe maggioritaria) ed entropia della distribuzione delle label. Cluster con purity ≈ 1 = cluster "puri".
- **c)** Caratterizzare i cluster con le feature: centroidi/mediane per cluster, heatmap delle feature (z-score) per cluster, top feature che distinguono un cluster dagli altri. Dare un nome "semantico" a ogni cluster (es. "flussi brevi, pacchetti UDP grandi, porta well-known" → famiglia di amplificazione).
- Cercare cluster misti benign+attacco: è traffico benigno con caratteristiche simili al malevolo (es. DNS legittimo vs DNS reflection)?

1.6.2 Anomaly detection su solo traffico benigno (punto 4.2 della traccia)

Strategia operativa

Split (punto chiave da spiegare bene):

- Benign diviso in *train* (es. 60%), *validation* (20%), *test* (20%), meglio se temporalmente.
- Tutti gli attacchi vanno **solo nel test** (non vengono mai visti).
- Scaler/PCA fittati **solo** sul benign di train.
- La soglia di anomalia si sceglie sul benign di validation (es. 95° o 99° percentile dello score ⇒ FPR atteso 5% o 1%), mai guardando gli attacchi.

Tecniche (almeno 2):

- **Isolation Forest**: veloce, poche assunzioni.
- **One-Class SVM** (kernel RBF): richiede scaling e un campione ridotto per costo computazionale; sensibile a `nu` e `gamma`.
- **Autoencoder**: si allena a ricostruire il benigno; score = errore di ricostruzione.
- **Local Outlier Factor** con `novelty=True`, oppure GMM (score = log-verosimiglianza).

Valutazione

- FPR sul benign di test; **detection rate per classe di attacco** (bar plot): risponde direttamente a "quali attacchi passano come normali".
- ROC e AUC (attacco vs benign) per confrontare le tecniche indipendentemente dalla soglia.
- Distribuzione degli score: istogramma benign vs ciascun attacco.
- Collegare i risultati al clustering: gli attacchi non rilevati sono quelli che finivano in cluster misti con il benigno?

Scelta finale: confrontare AUC, detection rate, FPR, tempo di training e facilità di tuning; motivare la scelta.

```
from sklearn.ensemble import IsolationForest
benign = df[df.label == "benign"]
attack = df[df.label != "benign"]
b_train, b_tmp = train_test_split(benign, test_size=0.4, random_state=42)
b_val, b_test = train_test_split(b_tmp, test_size=0.5, random_state=42)

scaler = StandardScaler().fit(b_train[FEATS]) # solo benign!
iso = IsolationForest(random_state=42).fit(scaler.transform(b_train[FEATS]))

score = lambda d: -iso.score_samples(scaler.transform(d[FEATS])) # alto = anomalo
thr = np.percentile(score(b_val), 99) # soglia da benign
test = pd.concat([b_test, attack])
test["pred_anom"] = score(test) > thr
print(test.groupby("label")["pred_anom"].mean()) # detection rate / FPR
```

1.7 Divisione del lavoro in 4 persone

1.7.1 Logica della divisione

Le sezioni non sono indipendenti: la Sezione 1 produce i dati per tutti, e la 4.1 dipende dalla 3. Una divisione "una sezione a testa" lascerebbe chi fa la Sezione 4 bloccato e chi fa la Sezione 1 senza lavoro dopo le prime settimane. La proposta bilancia il carico sfruttando i punti in cui il lavoro può partire in parallelo.

1.7.2 Ruoli

Persona	Responsabilità principale	Seconda fase
A – Data	Sezione 1 completa (EDA, pulizia, correlazione, PCA, scaling, encoding, feature matrix finale). Scrive `preprocessing.py`.	Sezione 2.3a (analisi FP/FN in termini di feature: chi conosce meglio le feature è la persona giusta). Coordina il report e l'uniformità dei grafici.
B – Classificazione	Sezione 2.1 e 2.2: i due split, training dei modelli, over/underfitting, tuning con CV, tabella tempo vs performance.	Sezione 2.3b e 2.3c (metriche nel contesto reale, confronto per-flow vs per-time).
C – Clustering	Sezione 3 completa: algoritmi, scelta di k/`eps`, metriche di similarità, tuning, visualizzazioni.	Supporto alla 4.1: fornisce a D le assegnazioni ai cluster salvate su file.
D – Anomaly & spiegabilità	Sezione 4.2 (anomaly detection): può partire subito dopo il preprocessing, non dipende dal clustering.	Sezione 4.1 (distribuzione delle classi nei cluster, caratterizzazione dei cluster) usando l'output di C.

1.7.3 Timeline indicativa

1. **Fase 0 – tutti insieme (pochi giorni).** Leggere traccia e paper CIC-DDoS2019, setup repo, prima esplorazione insieme, decisione condivisa su feature da rimuovere e gestione di `inf`/NaN. A consolida tutto in `preprocessing.py`.
2. **Fase 1 – lavoro parallelo.** A completa l'EDA e la PCA; B, C e D partono con la versione "base" del preprocessing (pulizia + log + scaling) senza aspettare la versione finale di A.
3. **Fase 2 – integrazione.** Si aggiorna tutto alla feature matrix finale di A. C consegna i cluster a D; A e B lavorano sulla 2.3.
4. **Fase 3 – report.** Ognuno scrive la propria parte; A uniforma; revisione incrociata (ognuno rilegge una sezione non sua).

1.7.4 Regole per lavorare bene insieme

- Un notebook per sezione, nessuno modifica il notebook di un altro; il codice condiviso sta in `src/`.
- Output intermedi salvati su file (`X_clean.parquet`, `splits.pkl`, `clusters.csv`) così le dipendenze non bloccano nessuno.
- Ogni scelta va annotata con la sua motivazione mentre la si fa: il report chiede continuamente "spiega perché".
- Breve riunione settimanale per allineare scelte che impattano gli altri (es. A cambia le feature ⇒ B, C, D devono saperlo).
- **Ognuno deve capire tutto il progetto**, non solo la propria parte: se è previsto un orale, le domande possono riguardare qualunque sezione. Fare una sessione finale in cui ciascuno spiega la propria parte agli altri.

1.8 Checklist finale

☐ Identificatori (IP, Flow ID, Timestamp) esclusi dalle feature.
☐ Scaler/PCA fittati solo sul train (pipeline).
☐ Stessi modelli su entrambi gli split.
☐ Macro-F1 e confusion matrix normalizzata, non solo accuracy.
☐ Tempi di training riportati per la scelta del modello.
☐ Label mai usate nel fit del clustering.
☐ Anomaly detection: attacchi solo nel test, soglia scelta sul benign.
☐ Ogni grafico ha titolo, assi con unità, scala log dove serve, legenda.
☐ Ogni domanda della traccia ha una risposta esplicita nel report.
☐ `random_state` fissato, risultati riproducibili.